



UNIVERSIDADE FEDERAL DE ITAJUBÁ

Instituto de Recursos Naturais

AquaGee Analytics: Sistema de detecção de precipitação.

Realizado por:

Natanael Silva Oliveira – natanaeloliveira2387@gmail.com

Orientação:

Prof. Dr. Enrique Vieira Mattos - enrique@unifei.edu.br
Prof. Dr. Michelle Simões Reboita - reboita@unifei.edu.br

Itajubá/MG 2025

SUMARIO

1.	INTRODUÇÃO	1
1.1	NOME DO PROJETO	2
1.2	OBJETIVO	2
2.	DESCRIÇÃO DO PRODUTO	2
3.	ESTRUTURA DOS DIRETÓRIOS DO PRODUTO.....	3
3.1	MAPAS INTERATIVOS	3
3.2	SÉRIES TEMPORAIS	6
3.3	COMPARAÇÕES	8
4.	ORGANIZAÇÃO NO GIT	9
4.1	GITHUB	9
4.2	DEPLOY DO PRODUTO	10
4.2.1	AUTENTICAÇÃO DO GEE	12
5.	CONSIDERAÇÕES FINAIS	18

1. INTRODUÇÃO

A precipitação é um dos principais elementos climáticos e sua variabilidade exerce forte influência sobre a agricultura, os recursos hídricos e a gestão de riscos ambientais. O presente trabalho tem como objetivo desenvolver e avaliar o AquaGEE Analytics (disponível em: <https://aquagee.streamlit.app/>), um dashboard interativo para análise de precipitação a partir de dados de satélite. A pesquisa utiliza os produtos GSMAP, IMERG e CHIRPS, integrados ao Google Earth Engine (GEE) e ao framework Streamlit em linguagem Python para disponibilizar informações sobre precipitação.

O sistema possibilita a visualização em mapas interativos e séries temporais na escala momentânea, diária, mensal e anual de precipitação. A ferramenta permite mostrar de forma rápida e intuitiva as estimativas de precipitação de satélites, possibilitando a identificação de anomalias associadas a eventos de grande escala.

Nas seções seguintes, são detalhados os componentes e funcionalidades do sistema, instruções de instalação, procedimentos operacionais e exemplos de relatórios e produtos gerados.

1.1 NOME DO PROJETO

AquaGee

1.2 OBJETIVO

O objetivo deste sistema foi elaborar uma ferramenta em python para a detecção de precipitação para o Brasil, utilizando a biblioteca streamlit e dados de satélite disponível no GEE (Google Earth Engine).

2. DESCRIÇÃO DO PRODUTO

O IMERG Late Run (Versão 07) da missão GPM. ele oferece uma resolução espacial de $0,1^\circ$ e temporal de 30 minutos, com cobertura a partir de junho de 2000. Oferece uma estimativa melhorada com uma latência de 14 horas após o tempo de observação, a banda utilizada foi a precipitation (mm h^{-1}).

O GSMaP, desenvolvido pela Agência de Exploração Aeroespacial do Japão (JAXA). Fornece estimativas de precipitação horária com a mesma resolução espacial de $0,1^\circ$. Sua variável hourlyPrecipitationRate (mm h^{-1}) é especialmente útil para o monitoramento de eventos e complementa as análises de curto prazo realizadas com o IMERG.

O CHIRPS devido à sua extensa série histórica, que se inicia em 1981. Duas versões foram utilizadas: o CHIRPS Daily, com dados diários e resolução de $0,05^\circ$, ideal para análises históricas com mais detalhes, e o CHIRPS Pentad, que consolida os dados em acumulados de 5 dias, ideal para agilizar o processamento dos produtos anuais. A banda utilizada para ambos foram precipitation (mm h^{-1} e mm pentad h^{-1}), sendo a versão pentadal utilizada preferencialmente na visualização do acumulado anual, mensal e nos cálculos da análise temporal, aproveitando sua natureza já agregada para otimizar o desempenho da plataforma. Como o CHIRPS combina observações de satélite com estações pluviométricas em superfície para gerar uma série temporal, os dados são apenas sobre o continente não abrangendo os oceanos.

Tabela 1: Comparativo entre os produtos de precipitação por satélite IMERG, CHIRPS e GSMaP.

Produto	Resolução espacial	Resolução temporal	Fonte
CHIRPS Pentad	0,05°	Pentada (5 dias)	ee.ImageCollection("UCSB-CHG/CHIRPS/PENTAD")
CHIRPS Daily	0,05°	1 Dia	ee.ImageCollection("UCSB-CHG/CHIRPS/DAILY")
IMERG (GPM)	0,1°	30 minutos	ee.ImageCollection("NASA/GPM_L3/IMERG_V07")
GSMaP	0,1°	1 Hora	ee.ImageCollection("JAXA/GPM_L3/GSMaP/v6/operational")

3. ESTRUTURA DOS DIRETÓRIOS DO PRODUTO

Os códigos do sistema estão armazenados em um repositório no GitHub (https://github.com/Natanael528/TCC_GEE), organizado para facilitar o desenvolvimento e manutenção. O repositório contém o arquivo principal da homepage, denominado 🏠 `pagina_inicial.py`, e diretórios principais, `pages`, contém os scripts referentes a cada página do sistema.

Dentro do diretório inicial, o arquivo chamado `requirements.txt` contém todas as bibliotecas usadas no código. É um arquivo essencial para que o código funcione.

3.1 MAPAS INTERATIVOS

Dentro do diretório “pages”, com o nome de [01_🌐Mapas Interativos.py](https://github.com/Natanael528/TCC_GEE) (https://github.com/Natanael528/TCC_GEE), O Módulo de Análise Espacial é dedicado à visualização de dados de precipitação em mapas interativos. O fluxo de processamento (Figura 3) é iniciado pela seleção de uma fonte de dados entre CHIRPS, IMERG ou GSMaP e de uma escala temporal. As opções de escala incluem "Última Imagem", "Acumulado Diário", "Acumulado Mensal" e "Acumulado Anual".

Para a opção "Última Imagem" (Figura 2), o sistema busca a imagem mais recente disponível na coleção selecionada. Para as análises de acumulados (Diário, Mensal e Anual), o sistema primeiramente filtra a coleção de imagens correspondente ao período de interesse definido pelo usuário.

O tratamento dos dados é específico para cada produto. Para o IMERG, cujos dados são fornecidos em taxa de precipitação (mm h^{-1}) a cada 30 minutos, cada imagem é multiplicada

por 0,5 para converter a taxa de precipitação em acumulado horário. Para o GSMap, que fornece dados horários (mm h⁻¹), o valor já representa o acumulado no intervalo de uma hora. Os dados do CHIRPS (diários ou em pântadas) já se encontram em valores acumulados. A imagem resultante é então renderizada em um mapa interativo na interface da aplicação por meio da biblioteca geemap, com uma legenda de cores que representa a intensidade da precipitação.

Fig 1 – Dicionário de datasets.

```
# Dicionário de datasets aprimorado com parâmetros de visualização #####ESCALA DA PALETA MM/DIA
DATASETS = {
    'CHIRPS': {
        'id': 'UCSB-CHG/CHIRPS/DAILY',
        'id2': 'UCSB-CHG/CHIRPS/PENTAD', # Usado para agregados maiores (mensal/anual)
        'band': 'precipitation',
        'multiplier': 1,
        'start_year': 1981,
        'name': 'CHIRPS',
        'vis_params': {
            'ultima_imagem': {'min': 0.1, 'max': 15, 'palette': PALETA_PRECIPITACAO},
            'diario': {'min': 0.5, 'max': 50, 'palette': PALETA_PRECIPITACAO},
            'mensal': {'min': 50, 'max': 600, 'palette': PALETA_PRECIPITACAO},
            'anual': {'min': 200, 'max': 3000, 'palette': PALETA_PRECIPITACAO},
        }
    },
    'IMERG': {
        'id': 'NASA/GPM_L3/IMERG_V07',
        'id2': 'NASA/GPM_L3/IMERG_V07', # Usa a mesma coleção para tudo
        'band': 'precipitation',
        'multiplier': 0.5, # Multiplicador para converter mm/30min para mm/dia (assumindo soma)
        'start_year': 2000,
        'name': 'IMERG',
        'vis_params': {
            'ultima_imagem': {'min': 0.1, 'max': 15, 'palette': PALETA_PRECIPITACAO},
            'diario': {'min': 0.5, 'max': 50, 'palette': PALETA_PRECIPITACAO},
            'mensal': {'min': 50, 'max': 600, 'palette': PALETA_PRECIPITACAO},
            'anual': {'min': 200, 'max': 3000, 'palette': PALETA_PRECIPITACAO},
        }
    },
    'GSMAP': {
        'id': 'JAXA/GPM_L3/GSMaP/v8/operational',
        'id2': 'JAXA/GPM_L3/GSMaP/v8/operational',
        'band': 'hourlyPrecipRate',
        'multiplier': 1, # Já está em mm/h
        'start_year': 2000,
        'name': 'GSMaP',
        'vis_params': {
            'ultima_imagem': {'min': 0.1, 'max': 15, 'palette': PALETA_PRECIPITACAO},
            'diario': {'min': 0.5, 'max': 50, 'palette': PALETA_PRECIPITACAO},
            'mensal': {'min': 50, 'max': 600, 'palette': PALETA_PRECIPITACAO},
            'anual': {'min': 200, 'max': 3000, 'palette': PALETA_PRECIPITACAO},
        }
    }
}
```

Fig 2 – Exemplo de uma das opções de seleção de aquisição do dashboard.

```
def ultima_imagem(info):
    """Mostra a imagem mais recente ou permite escolher entre as últimas disponíveis."""

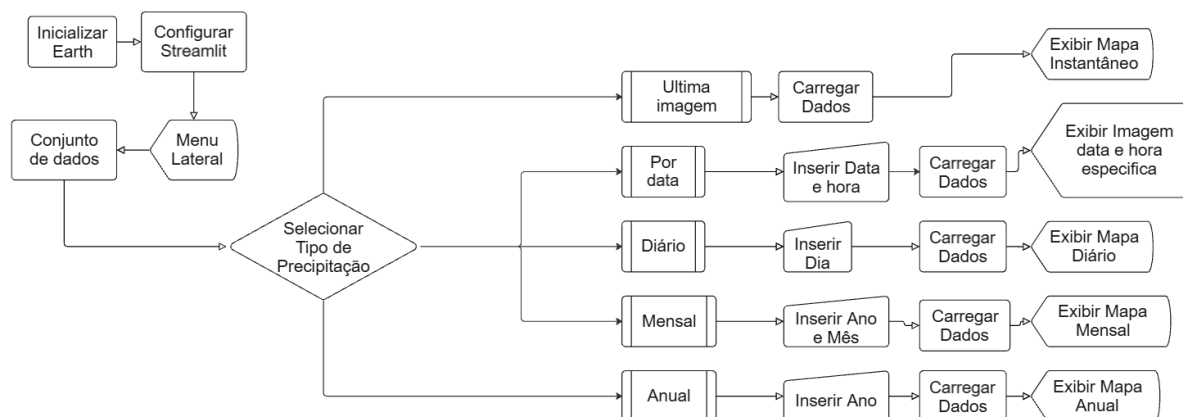
    ultima_data = get_ultima_data_disponivel(info)
    if not ultima_data:
        st.warning("Não foi possível encontrar imagens recentes com dados disponíveis.")
        return

    # Buscar imagens nos últimos 40 dias até a última data disponível
    inicio_busca = ultima_data - timedelta(days=40)

    # ----- >Função do Earth Engine para obter a coleção de imagens <-----
    colecao = (
        ee.ImageCollection(info['id'])
        .filterDate(str(inicio_busca), str(ultima_data + timedelta(days=1)))
        .sort('system:time_start', False)
    )
    # <----->

    imagens_disponiveis = colecao.aggregate_array("system:time_start").getInfo()
    if not imagens_disponiveis:
        st.warning("Nenhuma imagem encontrada.")
        return
    else:
        # Última imagem automaticamente
        img = colecao.first()
        if img is None:
            st.warning("Nenhuma imagem encontrada.")
            return
        data_img = ee.Date(img.get("system:time_start")).format("dd/MM/YYYY - HH:mm").getInfo()
    # Visualização
    vis = info['vis_params']['ultima_imagem']
    imagem_final = (
        img.select(info['band'])
        .multiply(info['multiplier'])
        .updateMask(img.select(info['band']).gt(vis['min']))
    )
    desenhar_mapa(
        imagem_final,
        vis,
        f"Imagem para a data - {data_img}",
        "Precipitação Instantânea"
    )
```

Fig 3 – Estrutura do código da página de mapas interativos.



3.2 SÉRIES TEMPORAIS

Da mesma forma que o módulo focos recentes, a geração dos *dashboards*, esta localizado no diretório “pages” com o nome de [02 !\[\]\(71ceb62b681518c82e95d615e7265d66_img.jpg\) Séries temporais.py \(Natanael528/TCC_GEE\)](#).

O Módulo de Análise Temporal permite extrair e visualizar séries temporais de precipitação para uma Região de Interesse (ROI) exemplificado na figura 5. O usuário pode escolher um Estado ou Município, definir um ponto com raio, criar um retângulo por coordenadas ou desenhar um polígono livre no mapa (figura 6). Também é possível selecionar a fonte de dados (CHIRPS, IMERG ou GSMaP) e o período desejado. Uma vez definidos os parâmetros, o sistema executa o processamento. Para gerar a série temporal, o sistema percorre cada unidade de tempo escolhida (figura 4) sendo dia, mês ou ano. Somando a precipitação do período e calcula a média espacial da ROI. Na climatologia mensal, os dados de todos os anos são agrupados por mês; depois se calcula a média anual da precipitação acumulada de cada mês.

Os dados processados voltam ao painel em forma de tabela e servem para calcular estatísticas simples, como identificar o ano mais seco e o mais chuvoso. Além disso, o sistema produz a série anual de precipitação, onde as barras mudam de cor para mostrar anomalias positivas ou negativas em relação à média histórica. Também é exibida a climatologia mensal, que revela o padrão sazonal de chuva da região.

Fig 4 – Função extração da precipitação diária.

```
def get_daily_precip(collection, roi, start_year, end_year, band_name, scale, multiplier):
    """
    Calcula série diária reduzindo cada imagem sobre a ROI.
    Faz verificação do número de imagens e aborta (lança RuntimeError)
    se a coleção exceder o limite seguro de getInfo (5000).
    """
    start = ee.Date.fromYMD(start_year, 1, 1)
    end = ee.Date.fromYMD(end_year, 12, 31).advance(1, 'day')
    daily_coll = collection.filterDate(start, end).filterBounds(roi)

    # Verifica tamanho da coleção para evitar getInfo massivo
    n_images = daily_coll.size().getInfo()
    if n_images > 5000:
        raise RuntimeError(f"too_many_images: coleção diária tem {n_images} imagens (>5000)")

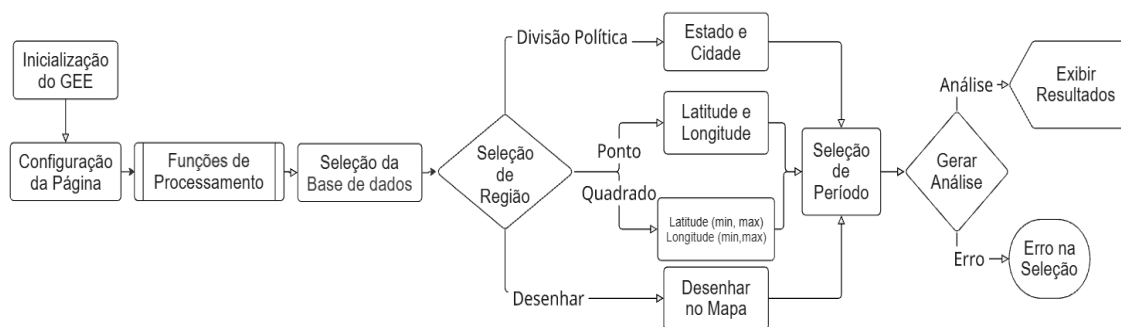
    def _per_image(img):
        img_band = img.select([band_name]).multiply(multiplier)
        date_str = ee.Date(img.get('system:time_start')).format('YYYY-MM-dd')
        val = img_band.reduceRegion(
            reducer=ee.Reducer.mean(),
            geometry=roi,
            scale=scale,
            maxPixels=1e13
        ).get(band_name)
        return ee.Feature(None, {'date': date_str, 'precip': val})

    daily_fc = daily_coll.map(_per_image)
    df = _fc_to_df(daily_fc)
    df = _ensure_date_and_precip(df, band_name=band_name, multiplier=multiplier)
    return df
```

Fig 5 – Código exemplo de como é feito o corte da area.

```
if tipo_analise == 'Por Divisão Política':
    tipo_divisao = st.sidebar.radio("Analisar por:", ('Município', 'Estado'))
    try:
        estados_info = collection_estados.aggregate_array('ADM1_NAME').getInfo()
        estados = sorted([estado for estado in estados_info if estado and estado != 'Name Unknown'])
        default_index = estados.index('Minas Gerais') if 'Minas Gerais' in estados else 0
        estado_selecionado = st.sidebar.selectbox("Escolha o Estado", estados, index=default_index)

        if tipo_divisao == 'Município':
            if estado_selecionado:
                with st.spinner("Carregando municípios..."):
                    municipios_filtrados = collection_municipios.filter(ee.Filter.eq('ADM1_NAME', estado_selecionado))
                    municipios = sorted(municipios_filtrados.aggregate_array('ADM2_NAME').getInfo())
                    municipio_selecionado = st.sidebar.selectbox("Escolha o Município", municipios, index=0)
                    if municipio_selecionado:
                        local_selecionado_nome = f"{municipio_selecionado}, {estado_selecionado}"
                        roi_fc = collection_municipios.filter(ee.Filter.And(ee.Filter.eq('ADM1_NAME', estado_selecionado), ee.Filter.eq('ADM2_NAME', municipio_selecionado)))
                        roi = roi_fc.geometry()
```

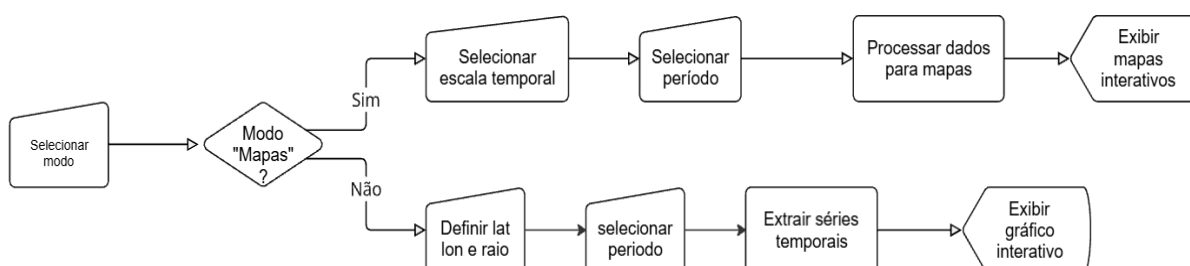
Fig 6 - Estrutura do código da página de séries temporais.

3.3 COMPARAÇÕES

Da mesma forma que o módulo “Séries Temporais” o código com o nome [03 Comparações.py](#) O Módulo de Comparações Visuais foi desenvolvido para facilitar a validação das estimativas de precipitação dos produtos. Ele funciona em dois modos: “Mapas” e “Gráfico” (Figura 7).

No modo “Mapas”, o usuário escolhe a escala temporal diária, mensal ou anual e define o período desejado. A partir disso, o sistema processa, em paralelo, os três conjuntos de dados para o mesmo intervalo, usando a mesma lógica de agregação adotada no Módulo de Análise Espacial. O resultado aparece como três mapas interativos lado a lado, cada um mostrando a distribuição espacial da precipitação estimada dos produtos.

No modo “Gráfico”, a comparação temporal é focada em um ponto específico. O usuário define a região de interesse circular indicando o centro (latitude e longitude) e o raio. Com esses parâmetros, a plataforma extrai as séries temporais diárias ou mensais para a mesma área nos três produtos. As séries são então reunidas em um único gráfico de linhas interativo, permitindo observar, no mesmo painel, onde as estimativas convergem e onde se afastam ao longo do tempo.

Fig 7 - Estrutura do script da página de Comparações.

Fonte: Elaborado pelo autor.ambém utiliza as informações que estão no diretório “dados” e concatena as planilhas para a plotagem do mapa da climatologia de focos de queimadas. Diferente dos outros códigos, essa página cria uma matriz de focos para que seja plotado o mapa de densidade de focos.

4. ORGANIZAÇÃO NO GIT

A biblioteca streamlit é de fácil acesso e muito intuitiva. Após a construção do código em seu ambiente remoto, para que o site seja funcional para o público, deve ser realizado o seguinte passo-a-passo.

4.1 GITHUB

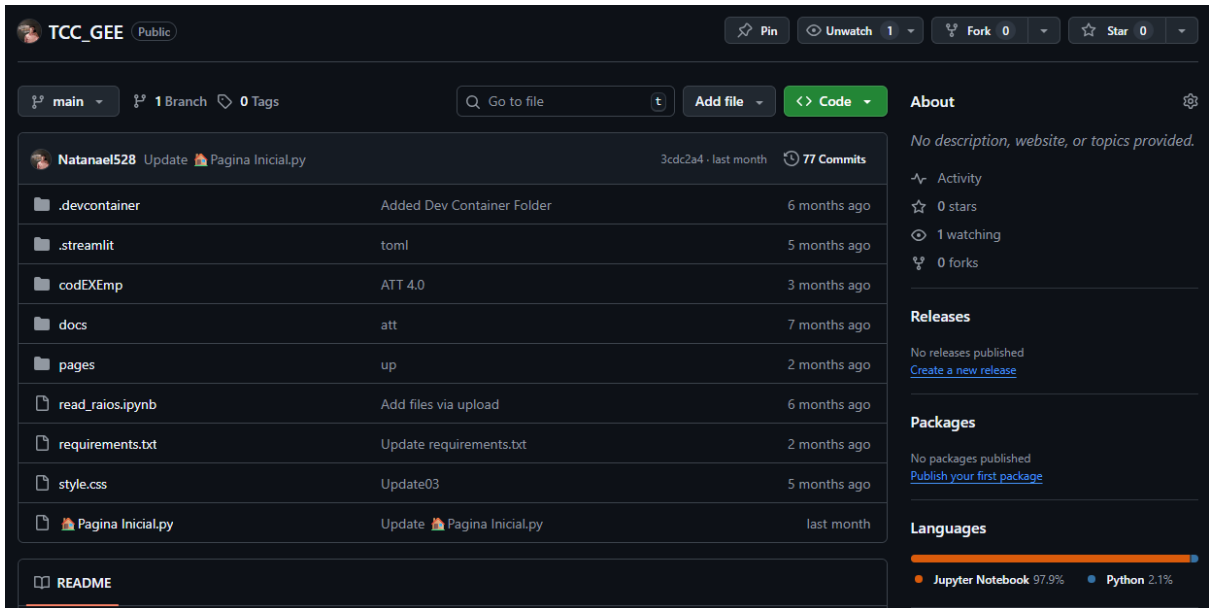
O processamento dos códigos é realizado através do Ambiente de Desenvolvimento Visual Studio Code (VSCODE) a estrutura de diretórios e arquivos é como mostrado na figura a seguir.

Fig 8 – Localização dos arquivos na pasta principal.

.devcontainer	17/07/2025 10:16	Pasta de arquivos	
.git	14/10/2025 15:46	Pasta de arquivos	
.streamlit	17/07/2025 11:58	Pasta de arquivos	
Analises	13/10/2025 15:47	Pasta de arquivos	
codEXEmp	02/10/2025 17:07	Pasta de arquivos	
docs	17/07/2025 10:16	Pasta de arquivos	
pages	24/09/2025 17:24	Pasta de arquivos	
read_raios	07/07/2025 12:20	Arquivo Fonte Jup...	3.306 KB
requirements	14/10/2025 15:39	Documento de Te...	1 KB
style	01/08/2025 18:35	Arquivo Fonte CSS	2 KB
🏠Pagina Inicial	14/10/2025 15:44	Arquivo Fonte Pyt...	12 KB

Após feito o código e organizado em uma pasta, deve se criar uma conta no GitHub, para que possa ser feito o commit do código no GitHub. Utilizando como referência a mesma estrutura de diretórios criada no computador, deverá ser montado no GitHub.

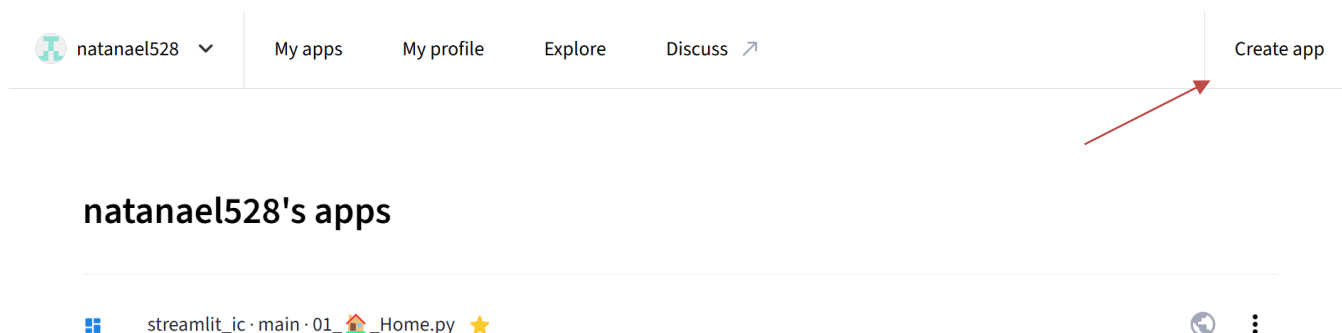
Fig 9 – Pasta anterior, agora dentro de um diretório do GitHub.



4.2 DEPLOY DO PRODUTO

Após ser realizado os passos anteriores, agora deverá ser realizado o deploy do produto no site do streamlit. Para isto deverá ser criado uma conta no site do streamlit, localizado em: <https://streamlit.io/>. Após criar a conta no streamlit, agora será realizado o deploy do produto. Para isso ao acessar o site do streamlit, clique na opção “Create app”, localizado no canto direto superior da página, como mostrado na figura a seguir.

Fig 10 – Homepage do streamlit. Seta vermelha indica a opção para a criação do App.

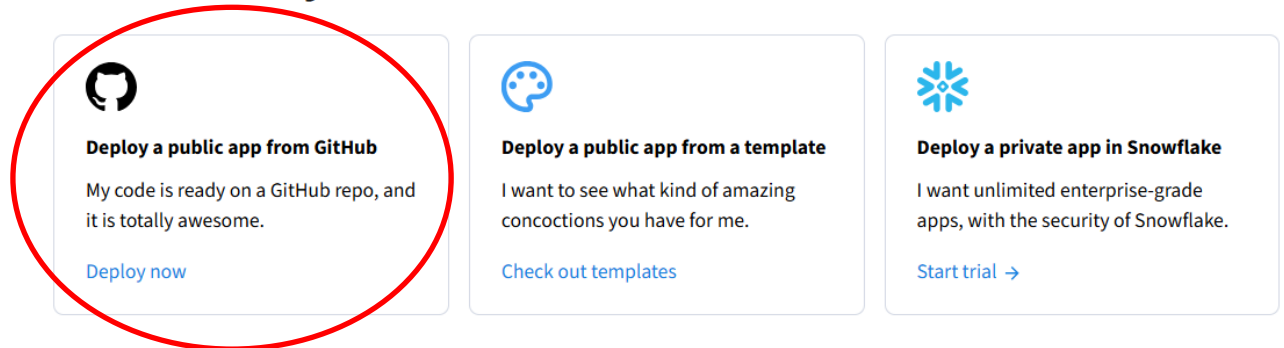


Em seguida deverá ser selecionado a opção “*Deploy a public app from Github*”.

Fig 11 – O círculo vermelho mostra a opção na qual deve ser selecionada.

[← Back](#)

What would you like to do?



The screenshot shows three deployment options:

- Deploy a public app from GitHub**
My code is ready on a GitHub repo, and it is totally awesome.
[Deploy now](#)
- Deploy a public app from a template**
I want to see what kind of amazing concoctions you have for me.
[Check out templates](#)
- Deploy a private app in Snowflake**
I want unlimited enterprise-grade apps, with the security of Snowflake.
[Start trial →](#)

Após esse passo iremos configurar as propriedades do deploy. Abaixo são listas que deverá ser preenchido.

- **Repository:** Deverá selecionar a localização do seu diretório do GitHub, onde estão os arquivos principais da aplicação.
- **Branch:** Manter como *main*.
- **Main file path:** Selecione o código que é o script principal. No nosso caso, é o código denominado “01_🏠_Home.py”.
- **App URL:** Defina o nome do link do seu App.

Após realizado as configurações, clique no botão azul “*Deploy*”. Pronto o deploy foi completado e o link do seu App já está liberado para funcionamento. Basta copiar o link e colar num browser de internet.

4.2.1 AUTENTICAÇÃO DO GEE

Passo a passo de como colocar a credencial na máquina local e no streamlit, para uso no streamlit.

1-Instalar as bibliotecas:

-streamlit

-earthengine-api

-geemap

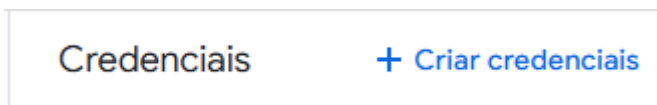
!pip install (nome da biblioteca)

2-Importar as bibliotecas em sua IDE de desenvolvimento (vscode, conda...)

```
import streamlit as st
import ee
import geemap.foliumap as geemap
```

3- Após a importação deveremos ir no site do gee para adquirir nossa credencial em:
<https://console.cloud.google.com/apis/credentials>

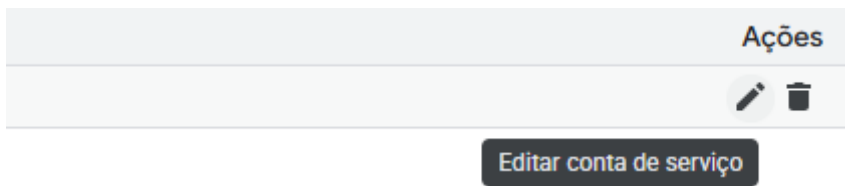
Clique em Criar credenciais,



Clique em “Conta de serviço” siga as instruções do site,



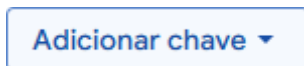
Clique no símbolo do lápis,



Clique em chaves,



Clique em adicionar chave, e crie uma chave,



Selecione JSON,

É feito o download de um arquivo contendo a chave privada. Armazene o arquivo com segurança porque essa chave não pode ser recuperada em caso de perda.

Tipo de chave

☒ JSON
Recomendado

☐ P12
Para compatibilidade com versões anteriores usando código com o formato P12

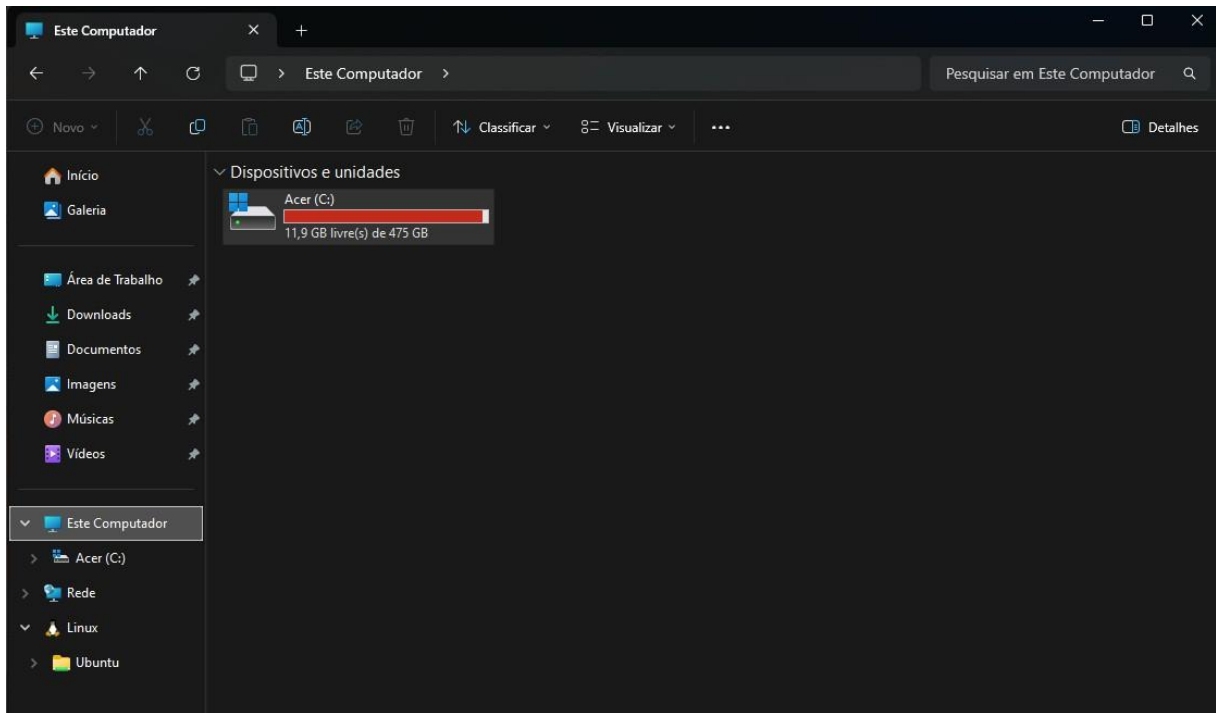
Cancelar Criar

Após ter clicado em criar, a chave JSON será baixada.

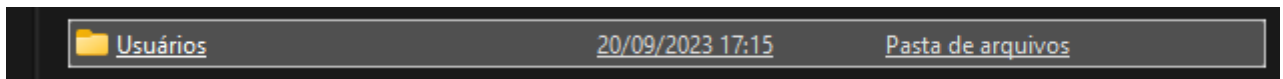
Pronto, esse arquivo contém as informações do seu projeto para a autenticação do GEE.

4- Após concluído o passo anterior, iremos colocar o arquivo no devido local para que funcione a autenticação do GEE.

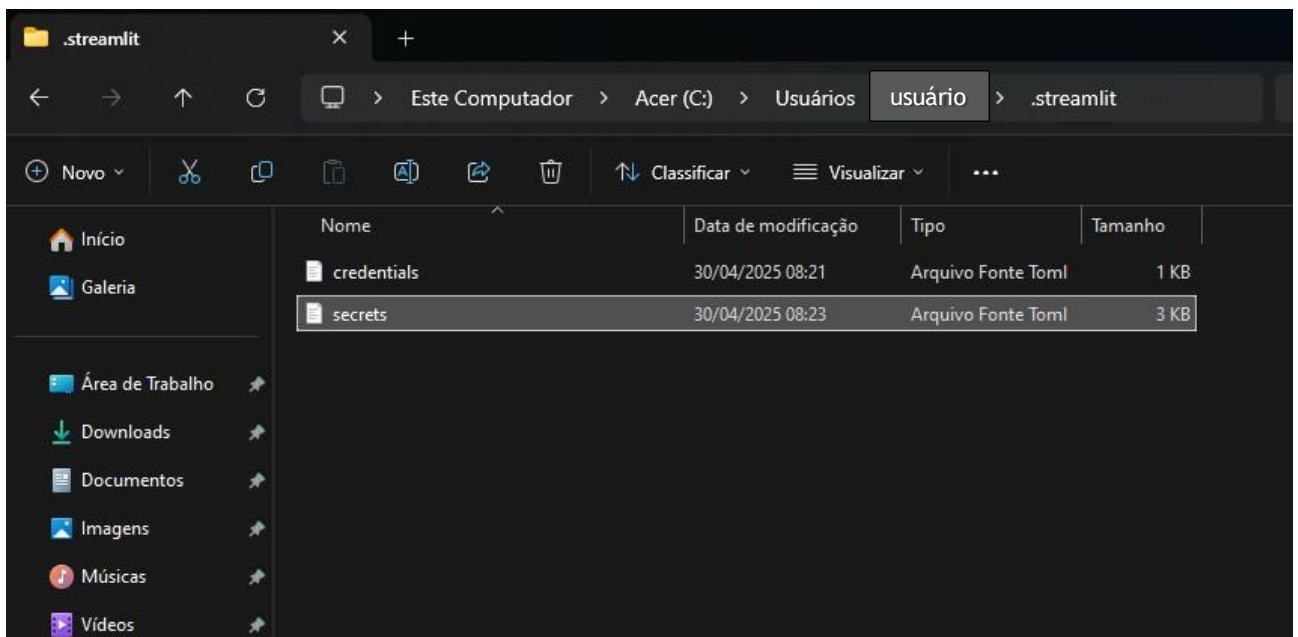
4.1- Autenticação no Streamlit na máquina local. Para isso devemos copiar o arquivo no devido local. Abra uma pasta qualquer.



Devemos achar a pasta USUÁRIOS



Dentro dessa pasta, ache o seu USUÁRIO, e procure as pasta “.streamlit”




```
# Cria arquivo temporário com as credenciais service_account_info =
dict(st.secrets["earthengine"])

with tempfile.NamedTemporaryFile(mode='w+', suffix='.json', delete=False) as f:
    json.dump(service_account_info, f)
    f.flush()

credentials = ee.ServiceAccountCredentials(service_account_info["client_email"],
f.name)

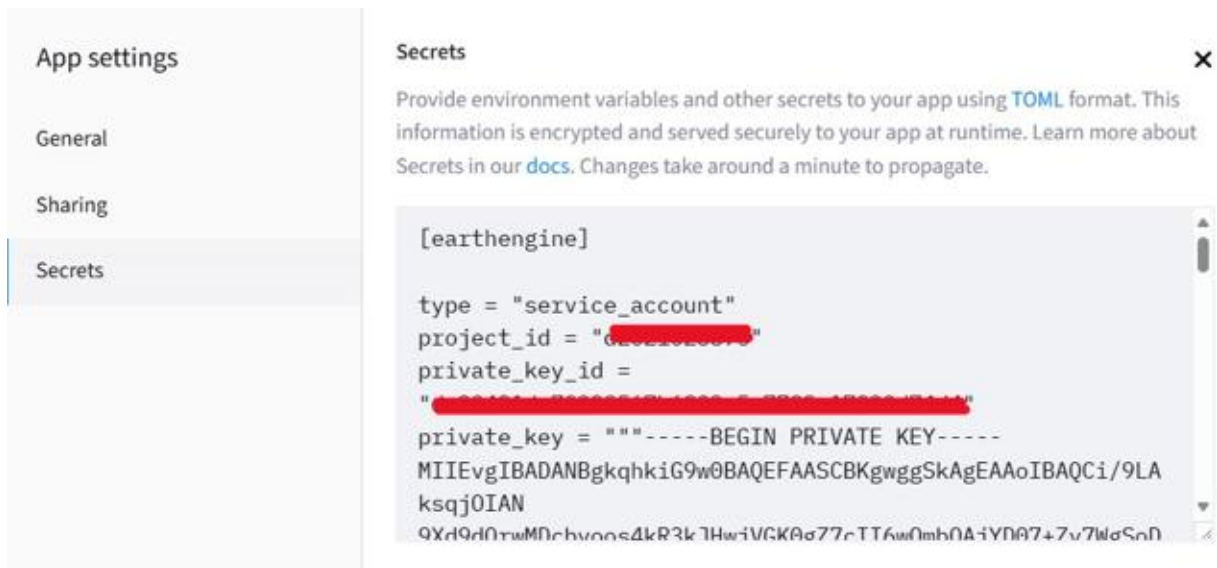
ee.Initialize(credentials)
```

4.2- Quando subir o código para o GIT e utilizar o site do streamlit para rodar o código, devemos fazer as seguintes alterações, para que o site consiga acessar as credenciais do GEE.

Clique em configurações (settings),

The screenshot shows the Streamlit Cloud interface for a user named 'natanael528'. The top navigation bar includes 'My apps', 'My profile', 'Explore', and 'Discuss'. Below the navigation bar, the user's apps are listed: 'streamlit_ic · main · 01_Home.py' and 'tcc_gee · main · home.py'. The 'tcc_gee' app is selected, and a context menu is open, showing options: 'Edit with Codespaces', 'Reboot', 'Delete', 'Analytics', and 'Settings'. The 'Settings' option is highlighted. Below the apps list, there is a section 'Get started from a template' with several templates: 'GDP over time', 'Chatbot', 'Existing tickets', and 'My new'. The 'App settings' modal is open, showing the 'General' tab. The 'App URL' field is set to 'geeteste.streamlit.app'. The 'Python version' is set to '3.12'. The 'Save changes' button is at the bottom right of the modal.

Cliquem em Secrets, e cole o conteúdo do arquivo que fizemos anteriormente.



Salve as configurações e pronto.

5. CONSIDERAÇÕES FINAIS

Este documento apresentou o desenvolvimento e funcionamento do AquaGee, um sistema de detecção de precipitação, projetado para atender à necessidade de monitoramento eficiente em território brasileiro. A ferramenta utiliza dados de satélites do fornecido pelo Google Earth Engine proporcionando informações atualizadas a além de possibilitar a análise histórica em diferentes escalas temporais e geográficas.

O uso da linguagem Python, aliado a bibliotecas robustas como Pandas, Plotly, Folium e Streamlit permitiu a criação de um dashboard intuitivo, dinâmico e de fácil utilização. A organização do código em um repositório GitHub acompanhada de um roteiro claro para atualização de dados, garante que a manutenção e a expansão do projeto possam ser realizadas de forma acessível e colaborativa.

ANEXO

Manual Didático do AquaGee.



AQUAGEE.pdf